

day10_statistics.py — full line-by-line walkthrough

MD Mutasim Billah · 52-week Data Science to ML/AI roadmap · Day 10, complete syntax and logic reference

Part 0 — imports, setup, and data loading

Imports

```
import os
import urllib.request

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
```

Each `import` line loads a library into memory so we can use its functions. `import os` gives us `os.path.exists()` later — checking if a file exists on disk. `import urllib.request` gives us a way to download a file from the internet if it's missing. `import numpy as np` loads NumPy and lets us call it as `np` instead of typing `numpy` every time — that's what `as` does, it's just a nickname. Same idea for `pandas as pd`, `matplotlib.pyplot as plt`, `seaborn as sns`. The last line, `from scipy import stats`, is different — instead of importing the whole `scipy` library, we're pulling out just the `stats` submodule, so we can write `stats.ttest_ind(...)` instead of `scipy.stats.ttest_ind(...)`.

Display options

```
pd.set_option("display.width", 120)
pd.set_option("display.max_columns", 20)
```

These two lines don't affect your data at all — they only change how pandas prints things to the terminal. `display.width` tells pandas to use up to 120 characters per line before wrapping, and `display.max_columns` says show up to 20 columns before truncating. Without these, wide DataFrames print ugly and cut off.

Constants

```
DATA_PATH = "titanic.csv"
DATA_URL = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
```

These are just variables holding text (strings). All-caps naming (`DATA_PATH` not `data_path`) is a Python convention meaning this value shouldn't change while the program runs — it's a constant, not enforced by Python, just a signal to humans reading the code.

The `load_titanic()` function

```
def load_titanic(path=DATA_PATH, url=DATA_URL):
```

`def` defines a function named `load_titanic`. The parentheses hold its parameters: `path` and `url`. The `=DATA_PATH` and `=DATA_URL` are default values — if you call `load_titanic()` with nothing, Python fills in those defaults automatically.

```
    if not os.path.exists(path):
```

`os.path.exists(path)` returns `True` or `False` depending on whether a file at that path is actually sitting on disk. `not` flips it — so this line reads as "if the file does NOT exist." Notice the indentation (4 spaces) — that's how Python knows this line belongs inside the `if` block, and inside the function.

```
        print(f"{path} not found locally – downloading...")
        urllib.request.urlretrieve(url, path)
```

The `f"..."` is an f-string — the `f` before the quote lets you drop a variable directly inside curly braces and Python swaps in its value. The next line, `urllib.request.urlretrieve(url, path)`, downloads whatever is at `url` and saves it to disk under the name `path`.

```
    df = pd.read_csv(path)
    return df
```

This runs whether or not the download happened. `pd.read_csv(path)` reads the CSV file off disk into a `DataFrame` (pandas' table object) — assigned to `df`. `return df` sends that `DataFrame` back out of the function so whoever called `load_titanic()` can use it.

Part 1 — descriptive statistics

describe_manual() — setup

```
def describe_manual(series):  
    """  
    Compute descriptive statistics by hand using plain Python/NumPy.  
    Returns a dict comparable to series.describe() plus variance/IQR/outliers.  
    """  
    data = series.dropna().to_numpy()  
    n = len(data)  
    sorted_data = np.sort(data)
```

`def describe_manual(series):` defines a function taking one parameter, a pandas Series (a single column). The triple-quoted text right after is a docstring — documentation, not executed code. `series.dropna()` removes missing values so they don't break the math, and `.to_numpy()` converts it into a plain NumPy array, which is faster to do math on. `len(data)` counts how many values are in the array. `np.sort(data)` returns a new array sorted ascending — needed later for the median and percentiles.

Mean

```
mean = data.sum() / n
```

`data.sum()` adds up every value in the array; dividing by `n` (the count) gives the average — the literal definition of mean.

Median

```
mid = n // 2  
median = sorted_data[mid] if n % 2 == 1 else (sorted_data[mid - 1] + sorted_data[mid]) / 2
```

The `//` operator is integer (floor) division — `mid` is the middle index. `n % 2 == 1` checks if `n` is odd using the modulo operator (the remainder after division). This line uses a ternary conditional expression: a value, then `if` a condition, then `else` a fallback value, all on one line. If `n` is odd, the single middle element is the median. If `n` is even, there's no single middle element, so we average the two middle values instead.

Mode

```
values, counts = np.unique(data, return_counts=True)  
mode = values[np.argmax(counts)]
```

`np.unique(data, return_counts=True)` returns two arrays at once: the distinct values in `data`, and how many times each one occurs. Assigning both at once into `values, counts` is called tuple

unpacking. `np.argmax(counts)` returns the index of the largest count, and `values[...]` uses that index to pull out whichever value appeared most often — the mode.

Variance and standard deviation

```
sq_diffs = (data - mean) ** 2
pop_variance = sq_diffs.sum() / n
sample_variance = sq_diffs.sum() / (n - 1)
pop_std = pop_variance ** 0.5
sample_std = sample_variance ** 0.5
```

`data - mean` subtracts a single number from every element of the array at once — this is NumPy broadcasting, no loop required. `** 2` squares each difference so negative and positive differences don't cancel out. Dividing the sum of squared differences by `n` gives population variance; dividing by `n - 1` instead gives sample variance — this is Bessel's correction, which corrects for a sample tending to slightly underestimate the true spread of a full population. `** 0.5` takes the square root, converting variance back into standard deviation, which is in the same units as the original data.

Percentiles (nested function)

```
def percentile(p):
    idx = (n - 1) * p
    lo = int(np.floor(idx))
    hi = int(np.ceil(idx))
    if lo == hi:
        return sorted_data[lo]
    frac = idx - lo
    return sorted_data[lo] + frac * (sorted_data[hi] - sorted_data[lo])
```

This defines a function inside a function — a nested function. `percentile` only exists inside `describe_manual` and can see its outer variables (`n`, `sorted_data`) without being passed them explicitly — that's called a closure. `idx = (n - 1) * p` computes a fractional index for a percentile like `p=0.25`. `np.floor(idx)` rounds down and `np.ceil(idx)` rounds up, giving the two nearest actual positions in the array; `int(...)` converts those to whole numbers since array indices must be integers. If the fractional index lands exactly on an integer (`lo == hi`), we return that value directly. Otherwise `frac` is how far `idx` sits between `lo` and `hi` (a fraction between 0 and 1), and the last line does linear interpolation: start at the value at `lo`, and move that fraction of the way toward the value at `hi`. This is the same method pandas and NumPy use internally, which is why our manual numbers match `.describe()` exactly.

Quartiles, IQR, and outlier fences

```

q1 = percentile(0.25)
q2 = percentile(0.50)
q3 = percentile(0.75)
iqr = q3 - q1

lower_fence = q1 - 1.5 * iqr
upper_fence = q3 + 1.5 * iqr
outliers = data[(data < lower_fence) | (data > upper_fence)]

```

Calling `percentile()` three times gives the first quartile (25th percentile), median (50th), and third quartile (75th). `iqr`, the interquartile range, is the spread of the middle 50 percent of the data — `q3` minus `q1`. The 1.5 times IQR rule is a standard convention for flagging outliers: anything below the lower fence or above the upper fence counts as unusually extreme. The last line is boolean masking — the condition inside the brackets builds a True/False array the same length as `data`, and the pipe symbol is the element-by-element or operator for arrays (the plain Python `or` doesn't work this way). Only elements where the mask is True get kept.

Returning the results

```

return {
    "count": n,
    ...
    "outlier_values": outliers,
}

```

Building and returning a dictionary — a set of key-value pairs. Each key like `"mean"` or `"iqr"` is a string label; each value is the number (or array) we just computed. Returning everything as one dict lets the caller access any stat by name, e.g. `result["mean"]`.

`run_descriptive_stats()` — looping and printing

```

for col in ["Fare", "Age"]:
    print(f"\n--- {col} (manual) ---")
    stats_dict = describe_manual(df[col])

```

`for col in [...]:` is a for loop that runs the indented block once per item in the list — first with `col = "Fare"`, then `col = "Age"`. `df[col]` selects that column from the DataFrame and passes it into the `describe_manual` function we just walked through.

```

for k, v in stats_dict.items():
    if k == "outlier_values":
        print(f"{k:>18}: {len(v)} values (showing first 5): {np.round(v[:5], 2)}")
    else:
        print(f"{k:>18}: {v:.4f}" if isinstance(v, (float, np.floating)) else f"{k:>18}: {v}")

```

`stats_dict.items()` gives every key-value pair in the dictionary; `for k, v in ...` unpacks each pair into `k` (the label) and `v` (the number). The format spec `{k:>18}` right-aligns text in a field 18

characters wide so labels line up in a neat column. The if/else branches: outlier values get special handling (count plus a rounded preview of the first 5 using array slicing `v[:5]`), otherwise we check with `isinstance()` whether the value is a decimal number — if so, format it to 4 decimal places; if it's a plain integer like count, print it as-is. This whole line is itself a ternary expression, the same pattern as the median calculation earlier.

```
return {col: describe_manual(df[col]) for col in ["Fare", "Age"]}
```

This is a dict comprehension — a compact way to build a dictionary in one line. It's equivalent to looping over the list and doing `result[col] = ...` each time, but written as a single expression. The function returns this combined dictionary so the stats can be reused later if needed.

Part 2 — distributions

Skew and kurtosis

```
fare_skew = df["Fare"].skew()
fare_kurt = df["Fare"].kurt()
age_skew = df["Age"].skew()
age_kurt = df["Age"].kurt()
```

`.skew()` and `.kurt()` are built-in pandas methods that compute skewness and kurtosis directly — no manual formula needed here, unlike the hand-rolled stats earlier. Skew measures asymmetry (a positive value means a long tail stretching to the right); kurtosis measures how heavy or light the tails are compared to a normal distribution.

Log transform

```
log_fare = np.log1p(df["Fare"]) # log1p handles Fare == 0 safely
log_fare_skew = log_fare.skew()
```

`np.log1p(x)` computes $\log(1 + x)$ instead of a plain $\log(x)$. Regular $\log(0)$ is undefined, but $\log1p(0)$ equals $\log(1)$, which is 0, so it safely handles any Fare values that are exactly zero. This is the log transform from the lesson — it compresses large values far more than small ones, pulling in a long right tail. Anything after a `#` on a line is a comment, ignored by Python, just a note for humans.

Building the 2x2 plot grid

```
fig, axes = plt.subplots(2, 2, figsize=(12, 9))
```

`plt.subplots(2, 2, ...)` creates a figure with a 2 by 2 grid of subplots (4 charts in one image) and returns two things at once — `fig`, the whole canvas, and `axes`, a 2 by 2 array of individual chart areas — unpacked with tuple unpacking again. `figsize=(12, 9)` sets the overall image size in inches.

Drawing each subplot

```
axes[0, 0].hist(df["Fare"].dropna(), bins=40, color="#378ADD", edgecolor="white")
axes[0, 0].set_title(f"Fare (raw) - skew={fare_skew:.2f}")
axes[0, 0].set_xlabel("Fare")
```

`axes[0, 0]` indexes into the grid to grab the top-left subplot (row 0, column 0). `.hist(...)` draws a histogram there; `bins=40` splits the data range into 40 buckets, and `color/edgecolor` control the bar fill and outline. `.set_title()` and `.set_xlabel()` label that specific subplot. This exact pattern repeats three more times for `axes[0, 1]` (log-transformed Fare), `axes[1, 0]` (raw Age), and `axes[1, 1]`, which instead uses `sns.kdeplot()` twice to overlay two smoothed density curves with a legend, comparing raw versus log-transformed Fare directly.

Saving the figure

```
plt.tight_layout()  
plt.savefig("day10_distributions.png", dpi=150)  
plt.close()
```

`plt.tight_layout()` automatically adjusts spacing so subplot titles and labels don't overlap.

`plt.savefig(...)` writes the current figure to a PNG file; `dpi=150` controls resolution (higher means sharper but bigger file). `plt.close()` frees the figure from memory — good practice once you're done with a plot, especially in a script that makes several.

Part 3 — probability basics

Marginal probability

```
n = len(df)
p_survived = df["Survived"].sum() / n
```

`len(df)` on a DataFrame gives the number of rows. `df["Survived"]` is a column of 0s and 1s (0 means died, 1 means survived). Summing a column of 0/1 values counts how many 1s there are — total survivors. Dividing by `n` gives the overall probability of survival, written $P(\text{Survived})$.

Conditional probability

```
n_female = (df["Sex"] == "female").sum()
p_survived_given_female = df.loc[df["Sex"] == "female", "Survived"].sum() / n_female
```

`df["Sex"] == "female"` creates a boolean Series (True/False for every row) — boolean masking again. Summing True/False values works because Python treats True as 1 and False as 0, so this counts how many female passengers there are. `df.loc[condition, "Survived"]` uses `.loc`, pandas' label-based selector, to filter rows where Sex is female, then grab only the Survived column from those filtered rows. Summing and dividing by `n_female` gives a conditional probability: $P(\text{Survived given Sex=female})$ — the probability of survival, given that we already know the passenger is female. The same pattern repeats right after for first-class passengers, giving $P(\text{Survived given Pclass=1})$.

Bayes' theorem

```
p_female = n_female / n

p_female_given_survived_bayes = (p_survived_given_female * p_female) / p_survived
```

`p_female` is the marginal probability of being female, with no condition attached — just the raw proportion. The next line is Bayes' theorem written directly in code: $P(\text{female given survived})$ equals $P(\text{survived given female})$ times $P(\text{female})$, divided by $P(\text{survived})$. It lets us flip a conditional probability around — we know $P(\text{survived given female})$ from the data directly, and use it to derive the reverse relationship without recounting anything.

Verifying Bayes directly

```
n_survived = df["Survived"].sum()
p_female_given_survived_direct = df.loc[df["Survived"] == 1, "Sex"].eq("female").sum() / n_survived
```

This computes the same quantity a second way, directly: filter to only survivors, look at their Sex column, and `.eq("female")` checks each value for equality to "female" (the same idea as `==`, written as a method so it can be chained onto the previous selection). Summing counts how many survivors were

female; dividing by total survivors gives $P(\text{female given survived})$ computed straight from the data, used to confirm Bayes' theorem lands on the same answer.

Part 4 — correlation vs causation

Correlation matrix

```
numeric_cols = ["Survived", "Pclass", "Age", "SibSp", "Parch", "Fare"]  
corr = df[numeric_cols].corr()
```

`df[numeric_cols]` selects multiple columns at once by passing a list of names instead of one string. `.corr()` computes the Pearson correlation coefficient between every pair of those columns, returning a square table where each cell shows how strongly two variables move together, from -1 (perfect opposite) to +1 (perfect together).

Heatmap

```
plt.figure(figsize=(7, 6))  
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0, square=True)
```

`plt.figure(...)` starts a fresh single chart, unlike the 2 by 2 grid from before. `sns.heatmap()` draws the correlation matrix as a color-coded grid: `annot=True` writes the actual number inside each cell, `fmt=".2f"` formats those numbers to 2 decimals, `cmap="coolwarm"` sets blue for negative and red for positive, `center=0` makes sure zero correlation sits at the neutral color in the middle of that scheme, and `square=True` keeps every cell perfectly square.

Part 5 — hypothesis testing

Splitting the groups

```
fare_survived = df.loc[df["Survived"] == 1, "Fare"].dropna()
fare_died = df.loc[df["Survived"] == 0, "Fare"].dropna()
```

Same `.loc` filtering pattern as before: split the Fare column into two separate groups — fares paid by people who survived, and fares paid by people who didn't — dropping any missing values from each group.

Running the t-test

```
t_stat, t_pvalue = stats.ttest_ind(fare_survived, fare_died, equal_var=False)
```

`stats.ttest_ind()` runs an independent two-sample t-test, comparing whether the means of two groups differ significantly. It returns two values at once — the t-statistic (how many standard errors apart the two group means are) and the p-value (the probability of seeing a difference this large if there were truly no difference) — unpacked into `t_stat` and `t_pvalue`. `equal_var=False` tells scipy not to assume the two groups have the same spread — this variant is called Welch's t-test, and it's the safer default when you're not sure the groups are equally variable, which is true here since survivors and non-survivors likely have very different fare spreads.

Interpreting the p-value

```
if t_pvalue < 0.05:
    print("Conclusion: p < 0.05, reject H0 — Fare differs significantly...")
else:
    print("Conclusion: p >= 0.05, fail to reject H0...")
```

Standard if/else branching on the p-value against the conventional significance threshold, alpha equals 0.05. If the p-value is below that threshold, the observed difference is unlikely to be due to random chance alone, so we reject the null hypothesis (H_0 : no real difference). Otherwise we don't have enough evidence to claim a real difference exists.

Chi-square test

```
contingency = pd.crosstab(df["Sex"], df["Survived"])
chi2, chi_pvalue, dof, expected = stats.chi2_contingency(contingency)
```

`pd.crosstab(df["Sex"], df["Survived"])` builds a contingency table — a grid counting how many passengers fall into every combination of Sex and Survived (how many females survived, how many males died, and so on). `stats.chi2_contingency()` runs the chi-square test of independence on that table, testing whether Sex and Survived are related or independent. It returns four values at once: the

chi-square statistic itself, the p-value, dof (degrees of freedom, tied to the table's size), and expected (what the counts would look like if the two variables were completely unrelated) — all unpacked in one line.

main() — tying it together

```
def main():
    df = load_titanic()
    run_descriptive_stats(df)
    run_distributions(df)
    run_probability(df)
    run_correlation(df)
    run_hypothesis_tests(df)

if __name__ == "__main__":
    main()
```

`main()` calls `load_titanic()` once, then runs each of the five analysis functions in order against the same `df`. Everything before this point in the file was just defining what each function does; this is where they actually run. The final two lines are a very common Python idiom: every file has a built-in variable called `__name__`. When you run a file directly, Python sets `__name__` to the string `"__main__"`. If this file were instead imported into another script, `__name__` would hold the module's name instead. This check means "only run `main()` if this file is being executed directly, not if it's being imported elsewhere" — a safety guard that keeps the file reusable as a library without automatically running the whole analysis.